

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

ISO 25010

Software quality								
Suitable	**Efficient**	**Compatible**	**Usable**	**Reliable**	**Secure**	**Maintainable**	**Flexible**	**Safe**
Complete	Time	Co-existence	recognizable	fault-free	confidential	modular	adaptable	operations
Correct	Resources	interoperable	learnable	availability	integrity	reusable	scalable	id risks
Appropriate	Capacity		user error	fault tolerant	non-repudiation	analysable	installable	fail-safe
			engagement	recoverability	accountability	modifiable	replaceable	hazard warn
			inclusivity		authenticity	testable		safe integration
			assistance		resistance			
			self-describe					

Risk

Risk = cost × likelihood

	cheap	expensive
unlikely	Accept	Transfer (insure)
likely	Reduce	Avoid

Security must be built-in from the start

- Complex and subtle
  - You cannot roll-your-own, must use standard solutions
  - If hodge-podge, weakest link determines security
- Security architecture
  - Follow standard
  - Moving target
    - Solns that were secure in 2023, are not in 2025
      - e.g.: RSA public key encryption, no longer secure
- Security design patterns
  - Out-of-scope for ASD

Secure architecture

Maturity level		Stream A Architecture Design	Stream B Technology Management
1	Insert consideration of proactive security guidance into the software design process.	Teams are trained on the use of basic security principles during design.	Elicit technologies, frameworks and integrations within the overall solution to identify risk.
2	Direct software design process toward known secure services and secure-by-default designs.	Establish common design patterns and security solutions for adoption.	Standardize technologies and frameworks to be used throughout the different applications.
3	Formally control the software design process and validate utilization of secure components.	Reference architectures are utilized and continuously evaluated for adoption and appropriateness.	Impose the use of standard technologies on all software development.

https://owaspsamm.org/model/design/secure-architecture/

Secure architecture

Maturity level		Stream A Architecture Design	Stream B Technology Management
1	Insert consideration of proactive security guidance into the software design process.	Teams are trained on the use of basic security principles during design.	Elicit technologies, frameworks and integrations within the overall solution to identify risk.
2	Direct software design process toward known secure services and secure-by-default designs.	Establish common design patterns and security solutions for adoption.	Standardize technologies and frameworks to be used throughout the different applications.
3	Formally control the software design process and validate utilization of secure components.	Reference architectures are utilized and continuously evaluated for adoption and appropriateness.	Impose the use of standard technologies on all software development.

Secure architecture

Maturity level		Stream A Architecture Design	Stream B Technology Management
1	Insert consideration of proactive security guidance into the software design process.	Teams are trained on the use of basic security principles during design.	Elicit technologies, frameworks and integrations within the overall solution to identify risk.
2	Direct software design process toward known secure services and secure-by-default designs.	Establish common design patterns and security solutions for adoption.	Standardize technologies and frameworks to be used throughout the different applications.
3	Formally control the software design process and validate utilization of secure components.	Reference architectures are utilized and continuously evaluated for adoption and appropriateness.	Impose the use of standard technologies on all software development.

https://owaspsamm.org/model/design/secure-architecture/

Secure software patterns



Mitre Top 25

Rank	ID	Name	Score	Cves in KEV	Rank vs. 2023
1	CWE-79	Cross-site scripting XSS	56.92	3	,1
2	CWE-787	Out-of-bounds write Buffer overrun	45.20	18	-1
3	CWE-89	SQL injection	35.88	4	0
4	CWE-352	Cross-site request forgery CSRF	19.57	0	,5
5	CWE-22	Path traversal ../../..	12.74	4	,3
6	CWE-125	Out-of-bounds read Buffer overrun	11.42	3	,1
7	CWE-78	OS command injection	11.30	5	-2
8	CWE-416	Use after free	10.19	5	-4
9	CWE-862	Missing authorization	10.11	0	,2
10	CWE-434	Dangerous file upload	10.03	0	0
11	CWE-94	Code injection	7.13	7	,12
12	CWE-20	Improper input validation	6.78	1	-6
13	CWE-77	Command injection	6.74	4	,3
14	CWE-287	Improper authentication	5.94	4	-1
15	CWE-269	Improper privilege management	5.22	0	,7
16	CWE-502	Deserialization of untrusted data	5.07	5	-1
17	CWE-200	Exposure of sensitive information to an unauthorized actor	5.07	0	,13
18	CWE-863	Incorrect authorization	4.05	2	,6
19	CWE-918	Server-side request forgery (SSRF)	4.05	2	0
20	CWE-119	Improper restriction of operations within the bounds of a memory buffer	3.69	2	-3
21	CWE-476	NULL pointer dereference	3.58	0	-9
22	CWE-798	Use of hard-coded credentials	3.46	2	-4
23	CWE-190	Integer overflow or wraparound	3.37	3	-9
24	CWE-400	Uncontrolled resource consumption	3.23	0	,13
25	CWE-306	Missing authentication for critical function	2.73	5	-5

Mitre Top 25 (Mitigated by secure languages Java/Rust)

Rank	ID	Name	Score	Cves in KEV	Rank vs. 2023
1	CWE-79	Cross-site scripting XSS	56.92	3	,1
2	CWE-787	Out-of-bounds write Buffer overrun	45.20	18	-1
3	CWE-89	SQL injection	35.88	4	0
4	CWE-352	Cross-site request forgery CSRF	19.57	0	,5
5	CWE-22	Path traversal ../../..	12.74	4	,3
6	CWE-125	Out-of-bounds read Buffer overrun	11.42	3	,1
7	CWE-78	OS command injection	11.30	5	-2
8	CWE-416	Use after free	10.19	5	-4
9	CWE-862	Missing authorization	10.11	0	,2
10	CWE-434	Dangerous file upload	10.03	0	0
11	CWE-94	Code injection	7.13	7	,12
12	CWE-20	Improper input validation	6.78	1	-6
13	CWE-77	Command injection	6.74	4	,3
14	CWE-287	Improper authentication	5.94	4	-1
15	CWE-269	Improper privilege management	5.22	0	,7
16	CWE-502	Deserialization of untrusted data	5.07	5	-1
17	CWE-200	Exposure of sensitive information to an unauthorized actor	5.07	0	,13
18	CWE-863	Incorrect authorization	4.05	2	,6
19	CWE-918	Server-side request forgery (SSRF)	4.05	2	0
20	CWE-119	Improper restriction of operations within the bounds of a memory buffer	3.69	2	-3
21	CWE-476	NULL pointer dereference	3.58	0	-9
22	CWE-798	Use of hard-coded credentials	3.46	2	-4
23	CWE-190	Integer overflow or wraparound	3.37	3	-9
24	CWE-400	Uncontrolled resource consumption	3.23	0	,13
25	CWE-306	Missing authentication for critical function	2.73	5	-5

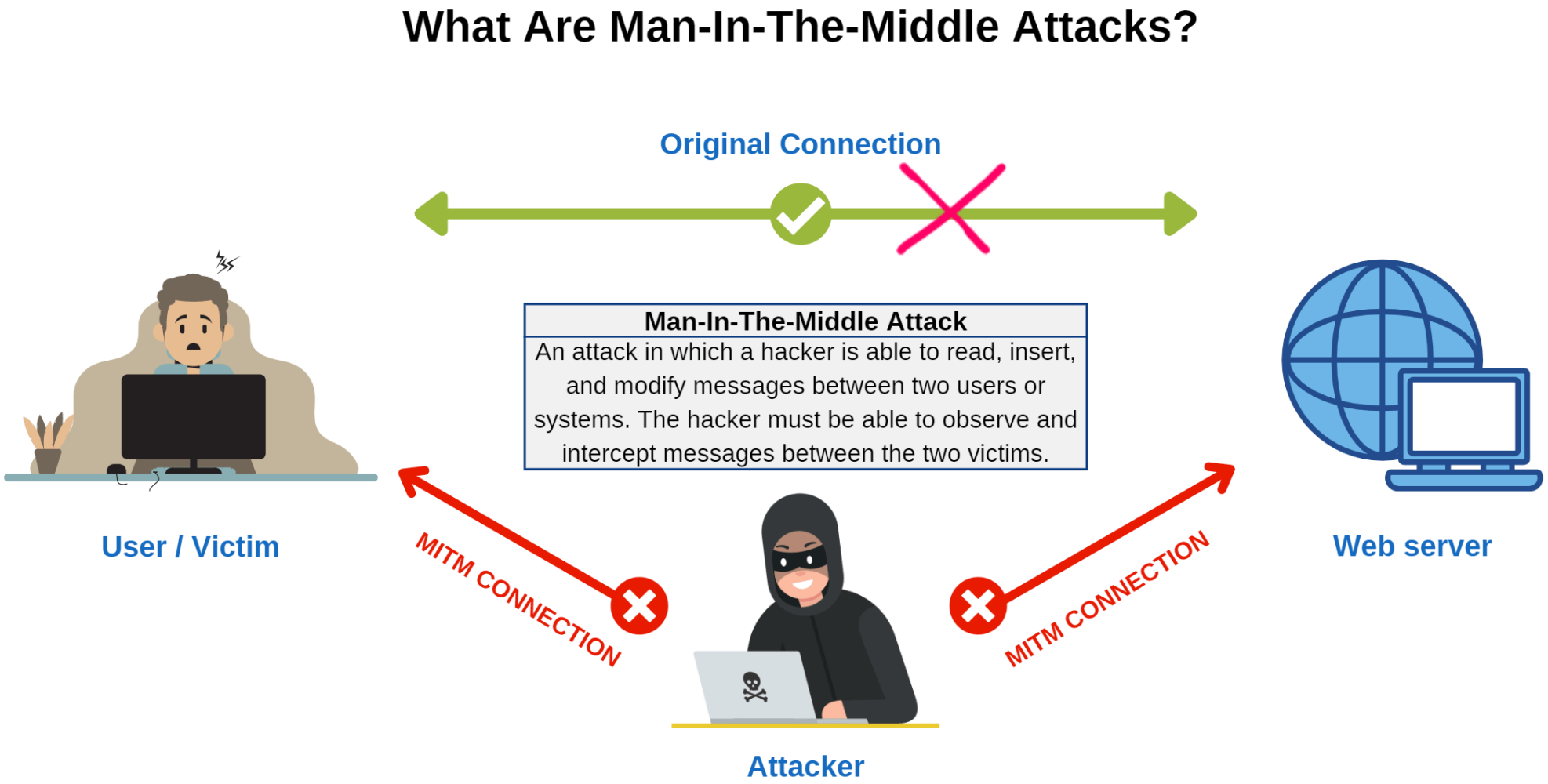
Mitre Top 25

Rank	ID	Name	Score
1	79, 352, 918	Cross-site tricks: scripting XSS, request forgery CSRF, SSRF	80.54
2	89, 78, 94, 77, 502	Injection: SQL, OS, Code, command	66.12
3	787, 125, 119	Out-of-bounds RW Buffer overrun, invalid buffer ops	60.31
4	287, 306, 269, 200, 862, 798	Improper/missing/hardcoded authentication	32.54
5	22	Path traversal ../../..	12.74
6	416	Use after free	10.19
7	434	Dangerous file upload	10.03
8	20	Improper input validation	6.78
9	476	NULL pointer dereference	3.58
10	190	Integer overflow	3.37
11	400	Uncontrolled resource consumption DoS	3.23

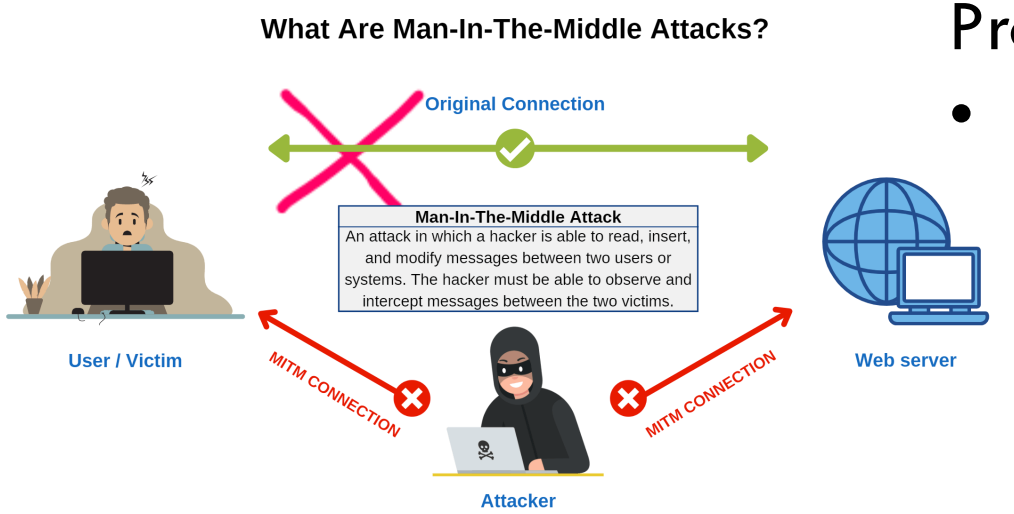
Security lingo

- Alice, Bob, Charlie: normal users, assumed to be noobs
- Eve: eavesdropper if RO, and Man-in-the-middle if R+W
- Mallory: Malicious actor, often a bot
- Trudy: Thief, tries to steal money/credentials

Man-in-the-middle



Man-in-the-middle



Problem:

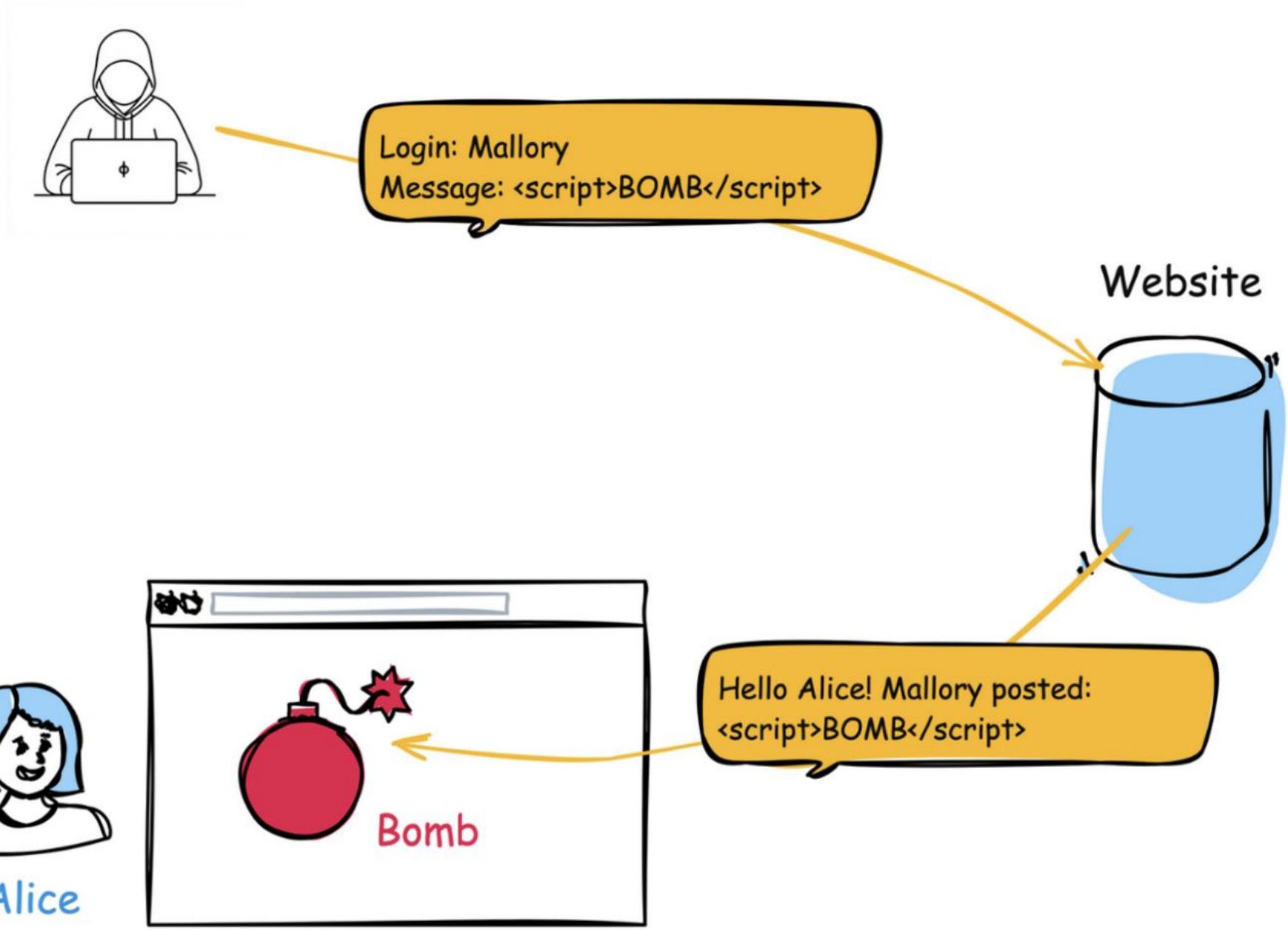
- Needs public key encryption
  - Pre 2023 algos not secure against quantum algorithms
  - Attacker can record sessions
  - Replay these when quantum algos are cheap enough

Mitigated by

- Use of secure encryption
- Latest standards
  - Must be secure against quantum computing
  - RSA, ECDSA, and EdDSA deprecated
  - Use: Post-Quantum encryption
    - ML-KEM, ML-DSA, and SLH-DSA

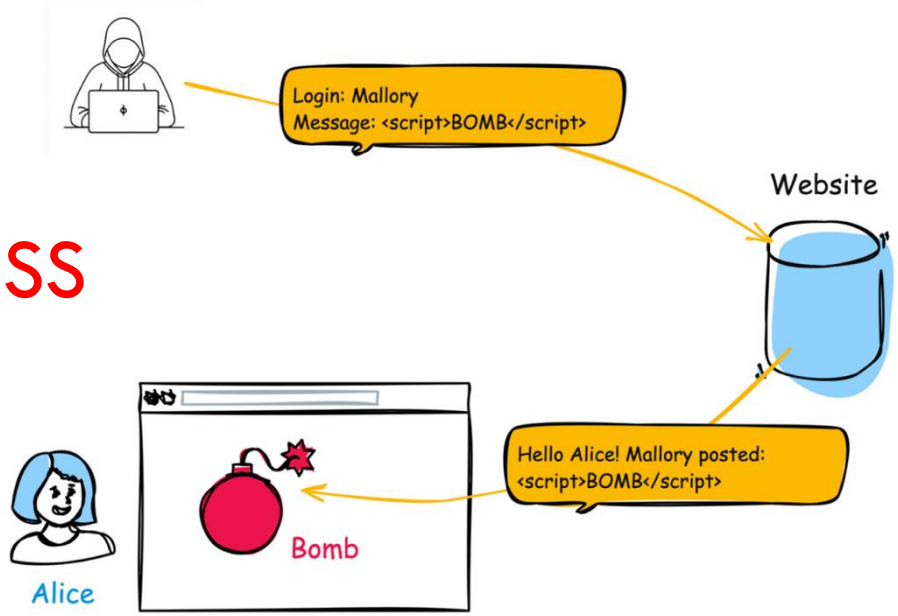
Cross-site scripting XSS

XSS



Cross-site scripting XSS

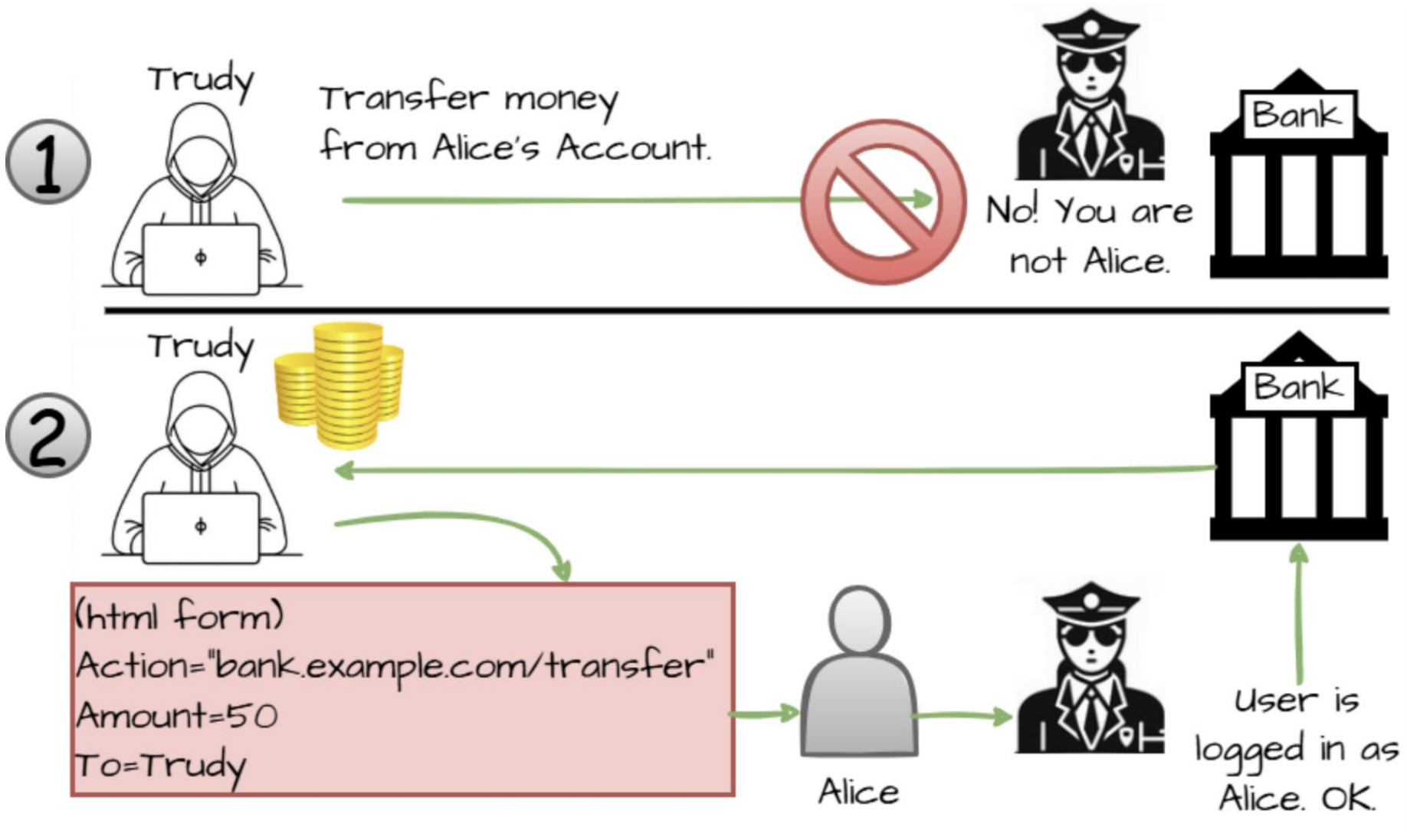
XSS



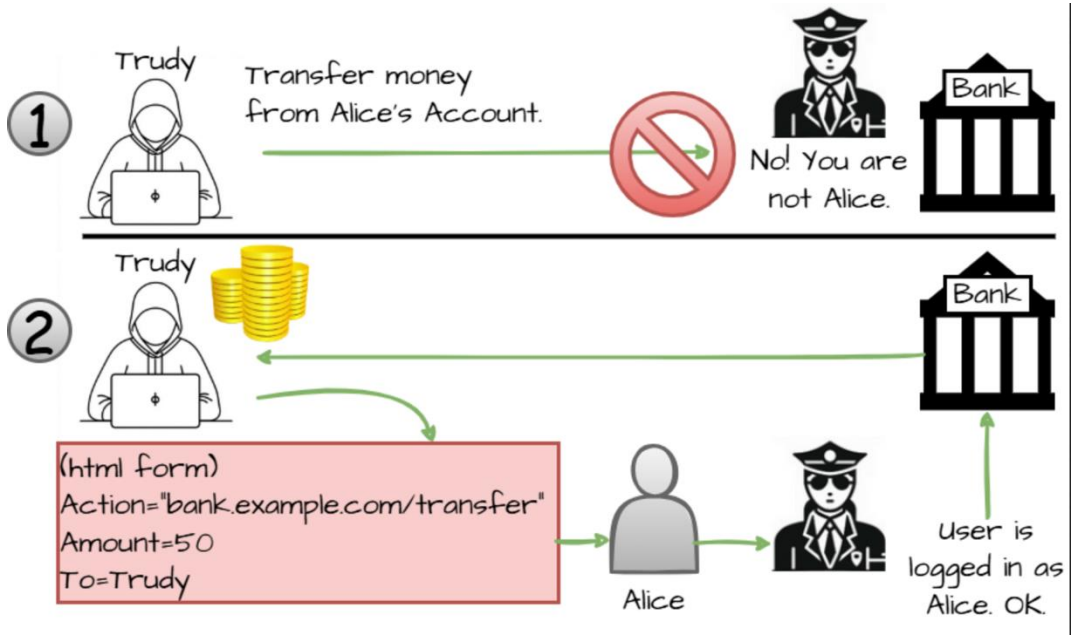
Mitigated by

- HTML sanitation
- HTML whitelisting (only allow safe subset)

Request forgery CSRF



Request forgery CSRF

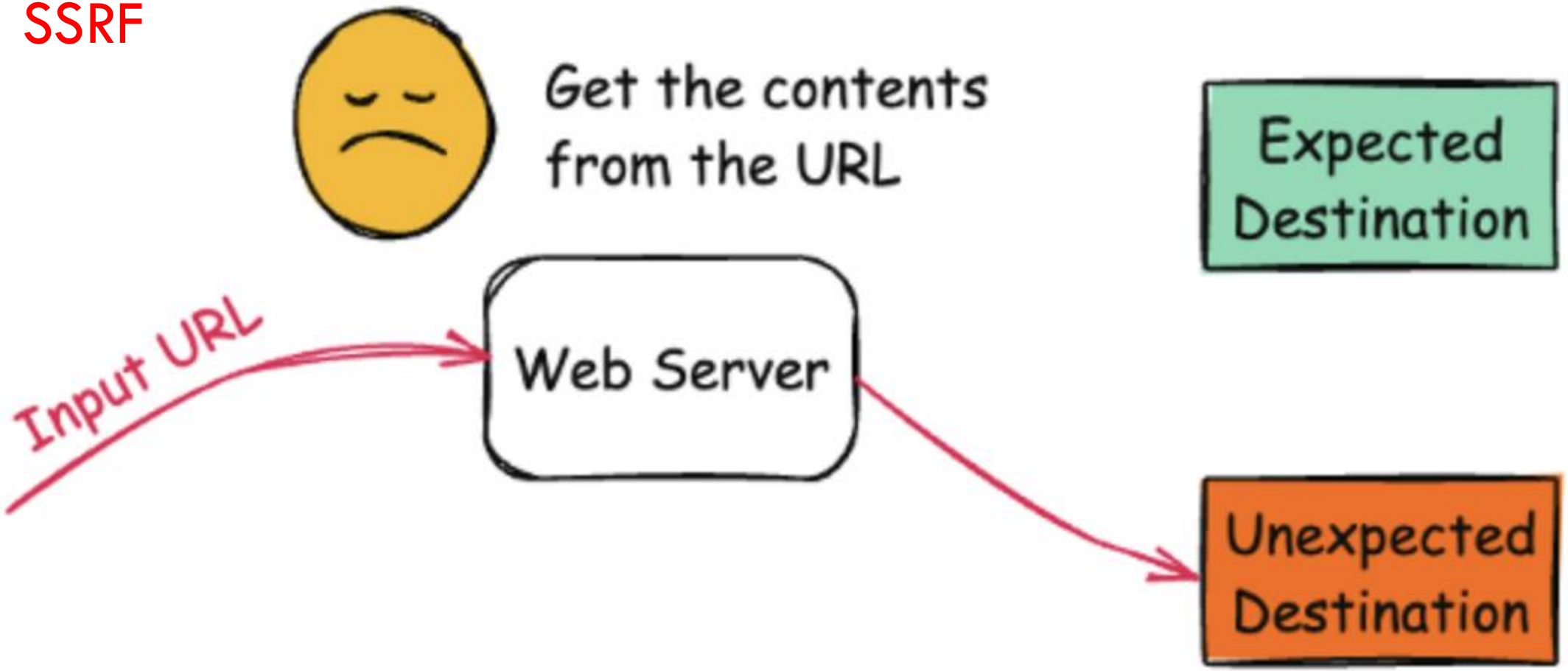


Mitigated by:

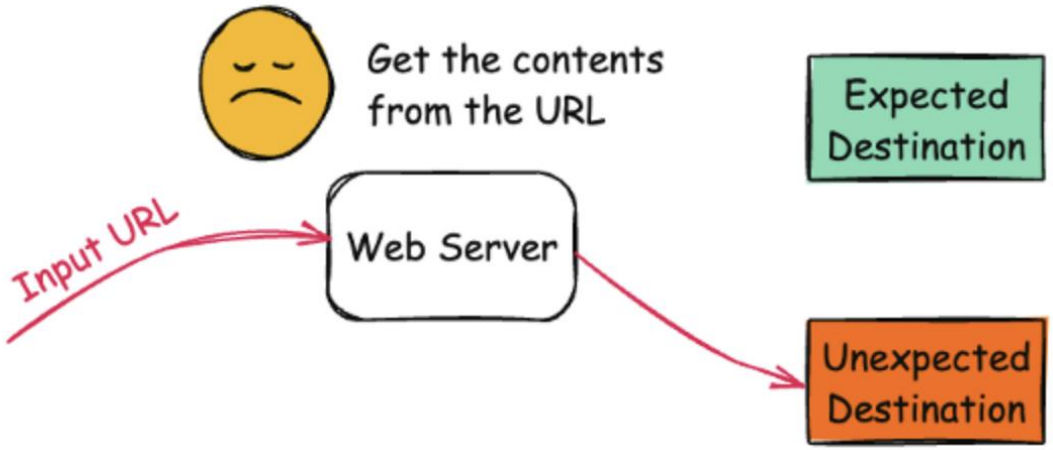
- CSRF Tokens
  - must not be part of cookies
  - must be unpredictable
  - unique token per session

Request forgery SSRF

SSRF



Request forgery SSRF



Mitigated by:

- It's complicated: read the SSRF Bible
- Input validation
- DNS hardening
- Do not give out internal IPs to external requests

Injection: SQL

SQL injection



Trudy

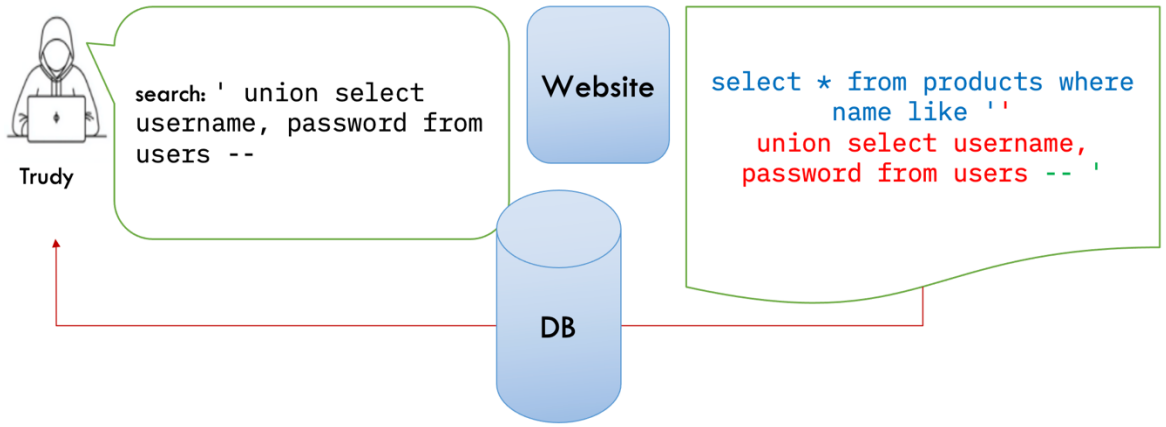
search: `' union select username, password from users --`

Website

```
select * from products where
name like ''
union select username,
password from users -- '
```

DB

Injection: SQL

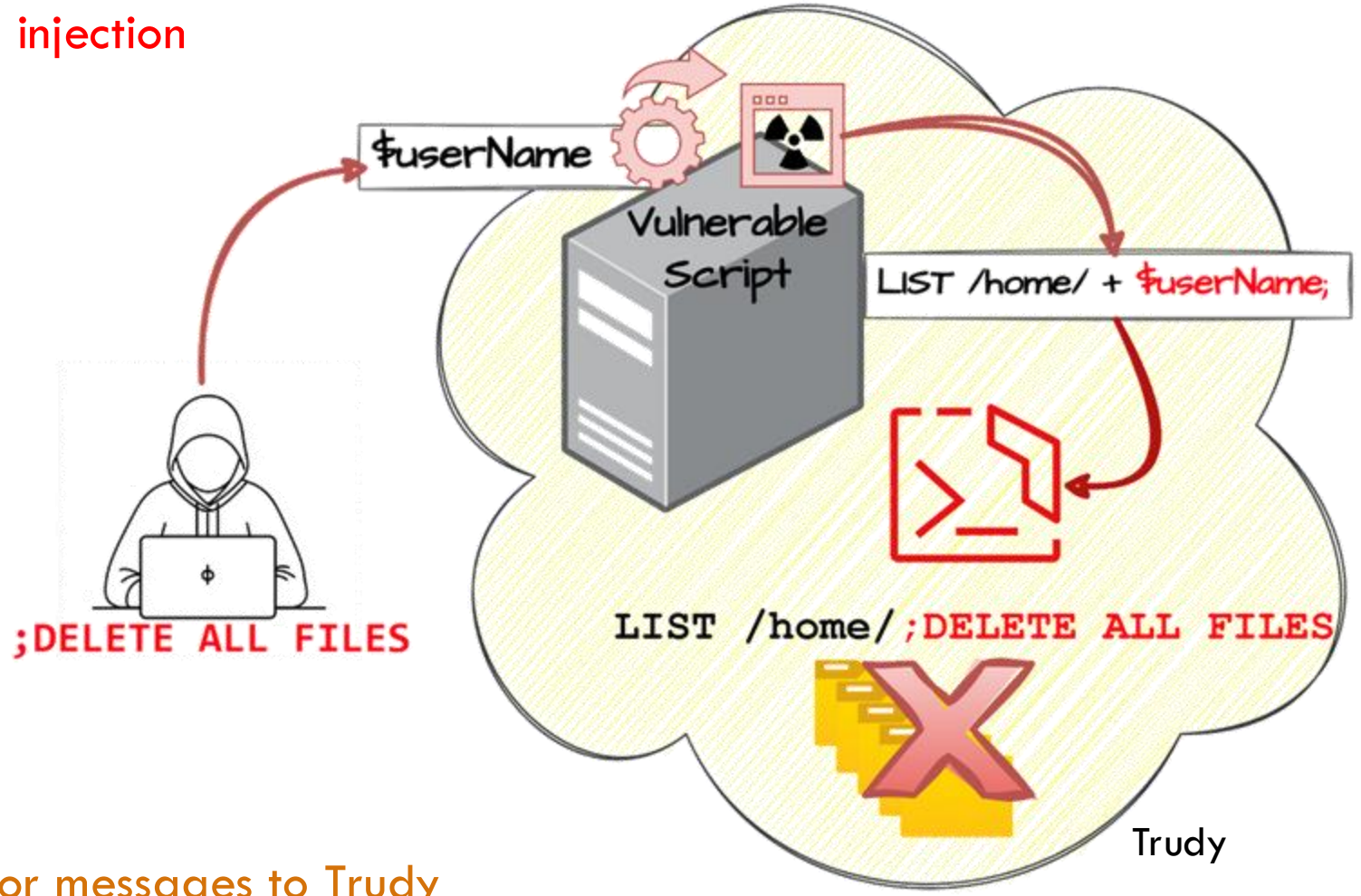


Mitigated by:

- Use parameterized queries
  - never copy-paste SQL statements
- If you have to use copy-paste, use a whitelist
  - E.g.: dynamic table names

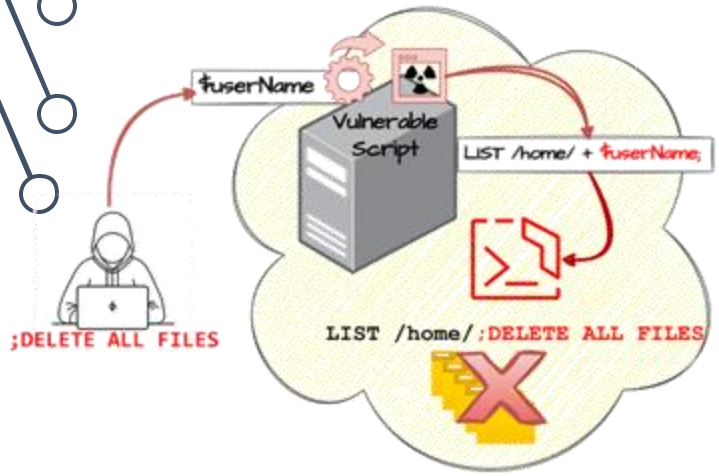
Injection: OS, Code, command

OS/code/command injection



Echoing verbose error messages to Trudy is very helpful for an attacker

Injection: OS, Code, command



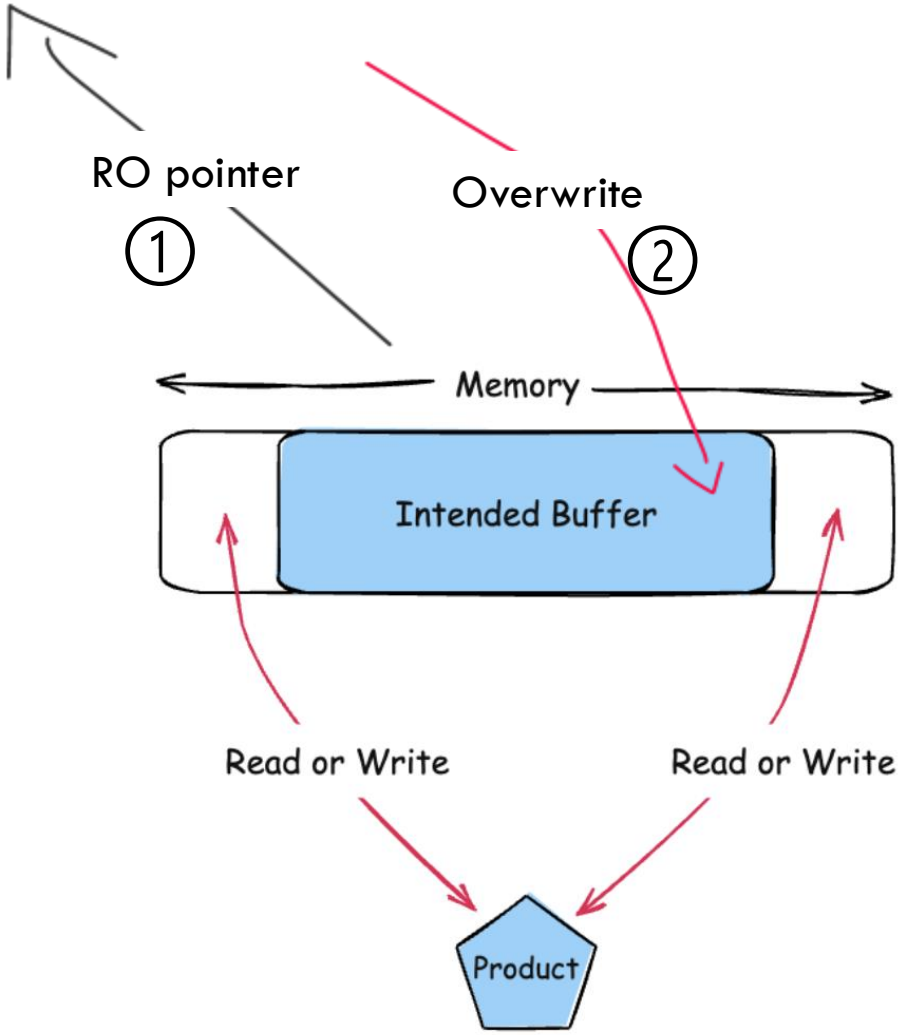
Mitigated by:

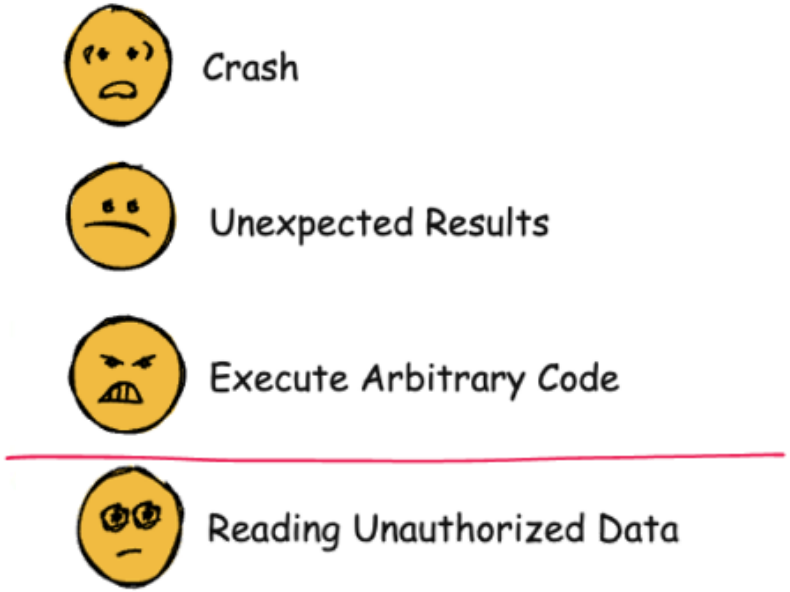
- Use parameterized queries
- Escape and sanitize all input
- Never copy-paste input into commands
- Do not echo error messages to users

Out of bounds R/W (Buffer overrun), invalid buffer ops

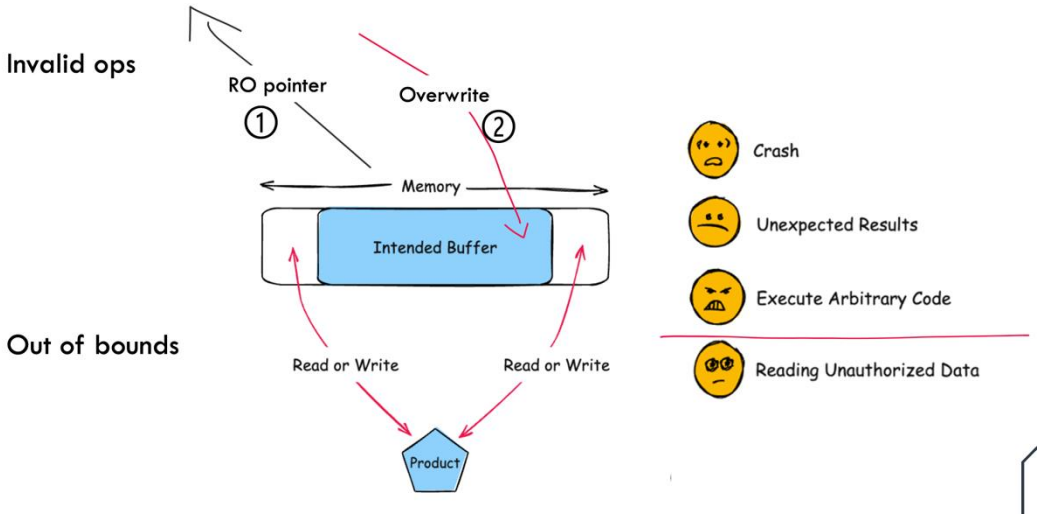
Invalid ops

Out of bounds





Out of bounds R/W (Buffer overrun), invalid buffer ops

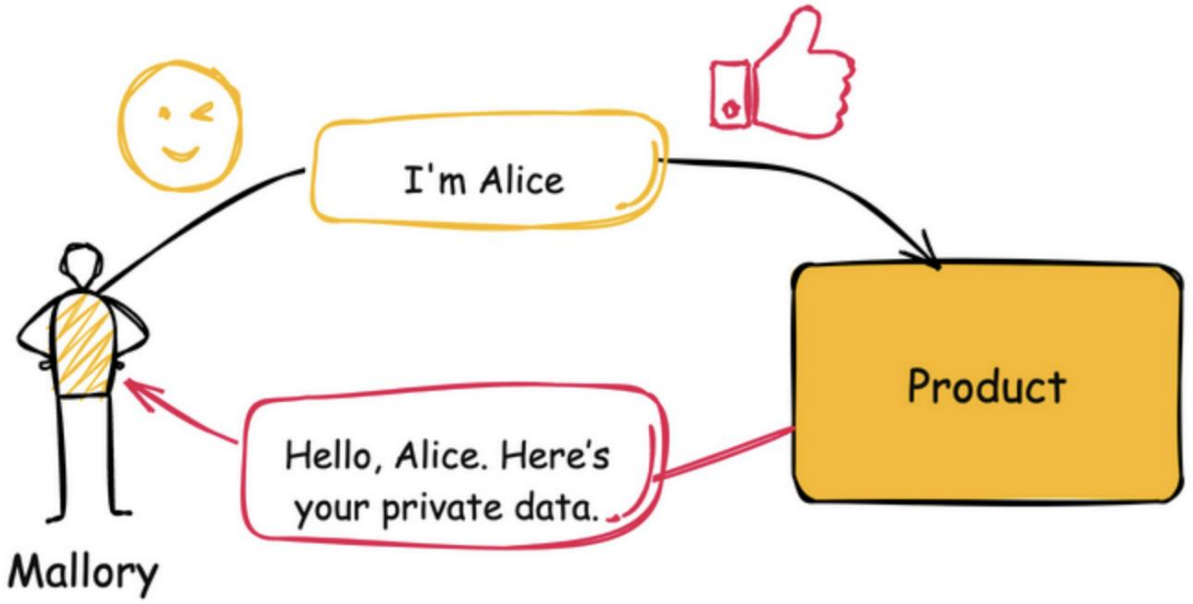


Mitigated by:

- C++/C: Use memory/address/undefined sanitizer
- Secure languages: Rust/Java
- Compiler hardening flags (stack canaries etc.)

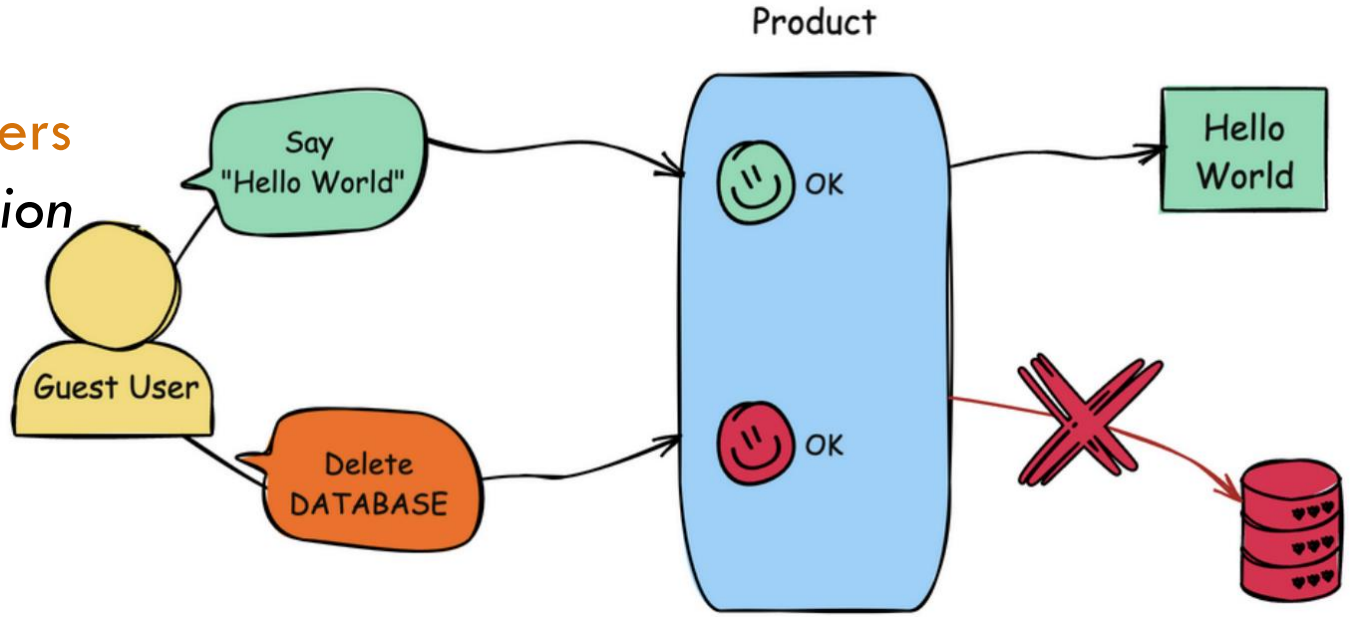
Improper/missing/hardcoded authentication

Weak/missing/hardcoded password or user-id in URL



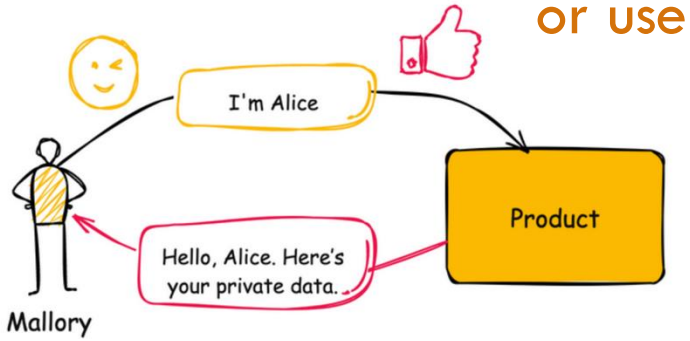
All users are superusers

often combined with injection



Improper/missing/hardcoded authentication

Weak/missing/hardcoded password
or user-id in URL



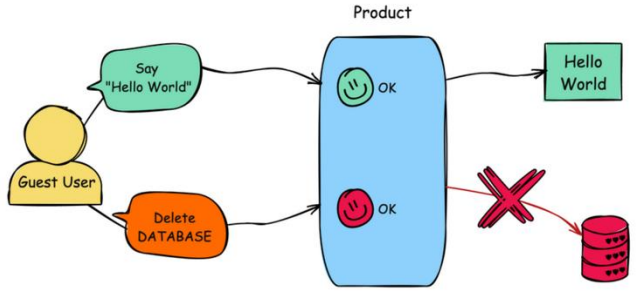
Mitigated by:

- Force change of default password after install
- Never put in backdoors
- Two-factor authentication
- Use standard (= known secure) authentication scheme

Improper/missing/hardcoded authentication

All users are superusers

often combined with injection



Mitigated by:

- Switch on brain (1960 is no more)
- Always disable guest users
- Sandbox untrusted visitors